MICROSOFT SENTINEL & DEFENDER FOR ENDPOINT — DoD IL5 / GCC HIGH / GCC

Automated USB (VID_03EB) Full Device Isolation & SOC Email Playbook

Customer Technical Deployment Guide • Portal ARM Paste, Manual Code View Paste & One-Shot Cloud Shell RBAC

Customer Setup Guide: Fast Portal Deployment (With or Without ARM)

This guide gives you **two fast, low-friction ways** to deploy in the Azure Portal by simply pasting JSON—no CLI/Bicep build pipelines required:

- **Fast Track (Portal ARM Paste — ~5 Minutes)**: Paste azuredeploy.json into **Deploy a custom template** in the Azure Portal to create the Analytics Rule, Logic App, Managed Identity, Sentinel Connection, and Automation Rule all at once, then run the 1-step permission grant in Phase 3.
- **Manual Portal Track (No ARM — ~10 Minutes)**: Follow Phases 1–4 below to create the Analytics Rule and Logic App in the Portal UI and paste portal-workflow-definition.json into the Logic App **Code view**.

Required Roles Checklist (Verify Before Starting)

Who	Required Role	Scope	Why It Is Needed
Deploying Admin (Azure)	`Owner` *(or* `Sentinel Contributor` + `Logic App Contributor` + `RBAC Administrator`*)*	Sentinel Resource Group	Creates the Analytics Rule, Logic App, Automation Rule, and assigns Azure RBAC roles.
Deploying Admin (Entra ID)	`Privileged Role Administrator` or `Global Administrator` *(Active in PIM)*	Entra ID Tenant	Grants `Machine.Isolate` and `Machine.ReadWrite.All` on `WindowsDefenderATP` (plus optional `Mail.Send` on `Microsoft Graph`) to the Logic App's Managed Identity. *(Note:* `Security Administrator` *and* `Azure Owner` *cannot grant Entra AppRoles).*
Deploying Admin (Exchange Online — Optional)	`Exchange Administrator` *(Only if scoping Graph* `Mail.Send`*)*	Exchange Online	Optional: Scopes the Logic App's `Mail.Send` permission to a single shared SOC mailbox via `New-ApplicationAccessPolicy` (or use the **Office 365 Outlook** connector in Designer with a mailbox sign-in instead).
Logic App Managed Identity *(Assigned in Phase 3)*	`Microsoft Sentinel Responder` + MDE `Machine.Isolate` & `Machine.ReadWrite.All` *(+ optional Graph* `Mail.Send`*)*	Resource Group + `WindowsDefenderATP` *(+* `Microsoft Graph`*)*	Reads Sentinel alerts, posts incident comments, isolates the endpoint in MDE, and sends optional email alerts.
`Azure Security Insights` *(Assigned in Phase 4)*	`Microsoft Sentinel Automation Contributor`	Resource Group	Allows the Sentinel Automation Rule to invoke the Logic App playbook.

Data Connector Prerequisite: In **Microsoft Sentinel > Data connectors > Microsoft Defender XDR**, verify that **raw** `DeviceEvents` **table streaming** is checked (not just *Incidents & Alerts*).

Fast Track: Paste `azuredeploy.json` in Portal ("Deploy a custom template")

If your change-control allows pasting an ARM template inside the Azure Portal UI, you can skip Phases 1 and 2 completely:

1. In the Azure Portal search bar, open **Deploy a custom template** -> click **Build your own template in the editor**.
2. Paste the entire contents of azuredeploy.json and click **Save**.
3. Select your **Subscription** and **Resource group**, enter your existing **Workspace Name**, and click **Review + create** -> **Create**.
4. In the Logic App -> **Development Tools > API connections**, open the Sentinel connection -> **Edit API connection** -> **Save** to authorize the Managed Identity connection.
5. Skip directly to **Phase 3 (Grant Permissions)** below!

Manual Portal Track (No ARM): 4 Consolidated Phases

Before starting: approve this containment policy and confirm Logic Apps Consumption, the Sentinel connector and MDE API are available and authorized in your DoD/IL5 boundary. A matching USB vendor ID does not prove maliciousness. This workflow requests **full isolation**: third-party forensic tools will not retain network access. Test the MDE proxy/VPN connection and release path first.

Phase 1. Create the Sentinel Detection Rule & Copy Resource IDs

In your existing Sentinel workspace, go to **Analytics > Create > Scheduled query rule**. Use these settings:

Setting	Value
Name	`USB VID_03EB connected - automatic full isolation`
Severity	High
Run query every	5 minutes
Look up data from last	1 hour
Alert threshold	Greater than 0 results
Event grouping	Trigger an alert for each event
Suppression	Off
Create incidents	On
Group related alerts into incidents	Off
Rule enabled	**Off until the pilot**

Paste this query:

KUSTO

```
DeviceEvents
| where ActionType == "PnpDeviceConnected"
| where coalesce(ingestion_time(), TimeGenerated) > ago(5m)
| extend UsbFields = parse_json(AdditionalFields)
| extend USBDeviceId = tostring(UsbFields.DeviceId),
         USBDescription = tostring(UsbFields.DeviceDescription),
         USBClassId = tostring(UsbFields.ClassId),
         USBClassName = tostring(UsbFields.ClassName)
| where USBDeviceId startswith @"USB\VID_03EB"
| where isnotempty(DeviceId)
| summarize arg_max(TimeGenerated, *) by DeviceId
| extend MDEDeviceId = DeviceId, DetectionSignature = "USB-VID-03EB-v1"
| project TimeGenerated, DeviceName, MDEDeviceId, USBDeviceId,
          USBDescription, USBClassId, USBClassName, DetectionSignature
```

Under **Alert enrichment > Custom details**, add these exact keys:

Key	Query column
`MDEDeviceId`	`MDEDeviceId`
`DetectionSignature`	`DetectionSignature`
`USBDeviceId`	`USBDeviceId`

Do not add a custom alert-title override. Keep the rule name above unchanged.

Save the rule. If the wizard creates it enabled, disable it immediately before

connecting automation. Do not enable incident grouping later: this workflow

intentionally processes only one matching alert/device per incident.

Copy two Resource IDs for Phase 2:

1. **Workspace Resource ID** (from Log Analytics workspace **Properties**):
`/subscriptions/SUBSCRIPTION-ID/resourceGroups/RESOURCE-GROUP/providers/Microsoft.OperationalInsights/workspaces/WORKSPACE-NAME`
2. **Analytics Rule Resource ID** (append `/providers/Microsoft.SecurityInsights/alertRules/<RULE-GUID>` using the rule GUID from the portal URL or exported rule JSON):
`/subscriptions/SUBSCRIPTION-ID/resourceGroups/RESOURCE-GROUP/providers/Microsoft.OperationalInsights/workspaces/WORKSPACE-NAME/providers/Microsoft.SecurityInsights/alertRules/ANALYTICS-RULE-GUID`

Phase 2. Create the Logic App, Enable Managed Identity & Paste Workflow JSON

1 **Create the Logic App & System-Assigned Identity**:

- Create a **Logic App (Consumption)** named `isolate-usb-03eb-dod` in the same resource group as your Sentinel workspace.
- Open **Identity > System assigned > On > Save** and copy the **Object (principal) ID** for Phase 3.

1 **Initialize the Managed Identity Sentinel Connection**:

- Open **Logic app designer**, add the **Microsoft Sentinel incident** trigger, and create its connection using **Managed identity** (never a personal user account). Click **Save**.

1 **Paste** `portal-workflow-definition.json` **into Code View**:

- Open portal-workflow-definition.json in a text editor and replace these two placeholders (keeping the JSON quotes):
- `PASTE-WORKSPACE-RESOURCE-ID`: full workspace Resource ID from Phase 1.
- `PASTE-ANALYTICS-RULE-RESOURCE-ID`: full analytics rule Resource ID from Phase 1.
- **Optional Email Alerts**:

- **Method A (Managed Identity Graph `sendMail` — No User Sign-in)**: Set `notificationSenderEmail`'s `defaultValue` to your sender mailbox UPN (e.g., `"soc-alerts@agency.mil"`) and `notificationRecipientEmail`'s `defaultValue` to your recipient address (e.g., `"soc@agency.mil"`). The playbook will automatically email on both Full Isolation Success (`Send_success_email_if_configured`) and Playbook Failure (`Send_failure_email_if_configured`).
- **Method B (Office 365 Outlook Connector in Designer — No Graph `Mail.Send` Needed)**: Leave `notificationSenderEmail` and `notificationRecipientEmail` blank (`""`), save Code view, switch to **Logic app designer**, and add **Office 365 Outlook -> Send an email (V2)** right under `Record_success` (and/or `Record_failure`).
- *(Note: If your rule name differs from the default, also update `alertTitle`'s `defaultValue` to match your exact rule name. If deploying in Commercial/GCC instead of DoD/GCC High, replace `https://management.usgovcloudapi.net` with `https://management.azure.com`, `https://management.core.usgovcloudapi.net/` with `https://management.core.windows.net/`, and `https://api-gov.securitycenter.microsoft.us` with `https://api.securitycenter.microsoft.com` or `https://api-gcc.securitycenter.microsoft.us`).*
- In the Logic App **Code view**, the document has a top-level `definition` object and a separate top-level `parameters` object:

1. **Keep the existing top-level `parameters` object**, especially `$connections.value`. It contains the Sentinel connection created by the portal.
2. Replace **only the value of `definition`** with the entire contents of portal-workflow-definition.json.
3. In the preserved `$connections.value`, the Sentinel connection key must be `azuresentinel`. If the designer generated another key, rename that key to `azuresentinel`, keeping its connection IDs/properties unchanged.
4. Click **Save**.

The outer document in **Code view** must keep this structure:

TEXT

```
{
  "definition": { contents of portal-workflow-definition.json },
  "parameters": { your existing portal-generated connection settings }
}
```

Do not replace the entire Code view document with the definition-only file, and do not replace your generated connection settings with an empty object.

Phase 3. Grant Azure RBAC & MDE Permissions (One-Time Admin Step)

You can grant all required permissions in **one shot** using Azure Cloud Shell (recommended, no extra modules or `Connect-MgGraph` consent prompts required), or via the Portal IAM UI + `Microsoft.Graph`.

Option 3A: One-Shot Cloud Shell Commands (`az cli` + `az rest` — Fastest)

Run this in Azure Cloud Shell (PowerShell) as an active **Global Administrator** or **Privileged Role Administrator** who also has **Owner** / **RBAC Administrator** on the Resource Group:

POWERSHELL

```
$subId       = "PASTE-SUBSCRIPTION-ID"
$rgName      = "PASTE-RESOURCE-GROUP-NAME"
$principalId = "PASTE-LOGIC-APP-IDENTITY-OBJECT-ID"
$graphUrl    = "https://dod-graph.microsoft.us" # Use https://graph.microsoft.us (GCC High) or
    https://graph.microsoft.com (GCC/Commercial)

# 1. Grant Microsoft Sentinel Responder to the Logic App Managed Identity
az role assignment create --assignee-object-id $principalId --assignee-principal-type
    ServicePrincipal `
    --role "Microsoft Sentinel Responder" --scope "/subscriptions/$subId/resourceGroups/$rgName"

# 2. Grant Microsoft Sentinel Automation Contributor to Azure Security Insights (so Automation
    Rules can run the playbook)
$sentinelSpId = az ad sp list --filter "appId eq '98785600-1bb7-4fb9-b9fa-19afe2c8a360'"
    --query "[0].id" -o tsv
az role assignment create --assignee-object-id $sentinelSpId --assignee-principal-type
    ServicePrincipal `
    --role "Microsoft Sentinel Automation Contributor" --scope
        "/subscriptions/$subId/resourceGroups/$rgName"

# 3. Auto-discover WindowsDefenderATP and grant Machine.Isolate + Machine.ReadWrite.All
$mdeSp = (az rest --method GET --url "$graphUrl/v1.0/servicePrincipals?`$filter=appId eq
    'fc780465-2017-40d4-a0c5-307022471b92'" | ConvertFrom-Json).value[0]
foreach ($perm in @('Machine.Isolate', 'Machine.ReadWrite.All')) {
    $roleId = ($mdeSp.appRoles | Where-Object { $_.value -eq $perm -and $_.isEnabled }).id
    @{ principalId = $principalId; resourceId = $mdeSp.id; appRoleId = $roleId } |
        ConvertTo-Json -Compress |
        az rest --method POST --url
            "$graphUrl/v1.0/servicePrincipals/$principalId/appRoleAssignments" --headers
            "Content-Type=application/json" --body "@-"
}

# 4. (Optional - if using built-in Managed Identity Graph Email) Grant Microsoft Graph Mail.Send
$graphSp = (az rest --method GET --url "$graphUrl/v1.0/servicePrincipals?`$filter=appId eq
    '00000003-0000-0000-c000-000000000000'" | ConvertFrom-Json).value[0]
$mailRoleId = ($graphSp.appRoles | Where-Object { $_.value -eq 'Mail.Send' -and $_.isEnabled
    }).id
@{ principalId = $principalId; resourceId = $graphSp.id; appRoleId = $mailRoleId } |
    ConvertTo-Json -Compress |
    az rest --method POST --url
        "$graphUrl/v1.0/servicePrincipals/$principalId/appRoleAssignments" --headers
        "Content-Type=application/json" --body "@-"
```

Optional DoD IL5 Step: Scope Graph `Mail.Send` to a Single Shared SOC Mailbox (`O365USGovDoD`)

In DoD IL5 environments, security teams often require restricting application-level `Mail.Send` so the Logic App's Managed Identity can **only** send mail from a single designated shared/service mailbox (e.g., `soc-alerts@agency.mil`) rather than any mailbox in the tenant:

1. **Option 1 — Restrict Graph `Mail.Send` with an Exchange Online Application Access Policy (`O365USGovDoD`)**:
Run in PowerShell as an **Exchange Administrator** (using `ExchangeOnlineManagement` 3.x+):

POWERSHELL

```
Connect-ExchangeOnline -ExchangeEnvironmentName O365USGovDoD

# Create a mail-enabled security group containing only the allowed SOC sender mailbox(es)
New-DistributionGroup -Name "Sentinel-Playbook-Allowed-Senders" `
    -PrimarySmtpAddress "sentinel-allowed-senders@agency.mil" -Type Security
Add-DistributionGroupMember -Identity "Sentinel-Playbook-Allowed-Senders" `
    -Member "soc-alerts@agency.mil"

# Restrict the Logic App Managed Identity (by its Application/Client ID) to ONLY that group
$logicAppClientId = (az ad sp show --id $principalId --query appId -o tsv)
New-ApplicationAccessPolicy -AppId $logicAppClientId `
    -PolicyScopeGroupId "sentinel-allowed-senders@agency.mil" `
    -AccessRight RestrictAccess `
    -Description "Restrict Sentinel USB Isolation Playbook to send mail only as
        soc-alerts@agency.mil"
```

1. **Option 2 — Use the `Office 365 Outlook` Connector in Logic App Designer (Zero Graph Permissions)**:

- If your Entra ID / Exchange team prefers not to grant `Mail.Send` on Microsoft Graph at all, leave `notificationSenderEmail` and `notificationRecipientEmail` blank (`""`).
- After saving the Logic App, open **Logic app designer**, expand `Process_incident` -> `Send_success_email_if_configured` (or add an action right after `Record_success`), click **Add an action**, search for **Office 365 Outlook** -> **Send an email (V2)**, and sign in with the dedicated SOC service/shared mailbox account.

Fault-Tolerant Email Design: If MDE Full Isolation succeeds (`Record_success`) but the email notification fails (for example, `Mail.Send` has not been granted yet in PIM or the sender UPN has no Exchange mailbox), the playbook catches the email error in `Record_email_warning`, posts a warning comment to the Sentinel incident, and still completes the Logic App run as `Succeeded` so MDE containment is never falsely reported as failed.

Option 3B: Portal IAM UI + `Microsoft.Graph` PowerShell

If you prefer assigning Azure RBAC in the Portal and using `Microsoft.Graph` on a workstation:

1. On the Log Analytics workspace (or Resource Group), go to **Access control (IAM) > Add role assignment**, select **Microsoft Sentinel Responder**, assign to **Managed identity**, and select your Logic App.
2. In PowerShell 7 with `Microsoft.Graph.Applications` installed (`Install-Module Microsoft.Graph.Applications -Scope CurrentUser`), locate the **WindowsDefenderATP** Enterprise Application Object ID and run:

POWERSHELL

```
$ErrorActionPreference = 'Stop'
$tenantId = 'PASTE-TENANT-ID'
$identityId = 'PASTE-LOGIC-APP-IDENTITY-OBJECT-ID'
$mdeApiObjectId = 'PASTE-MDE-API-ENTERPRISE-APP-OBJECT-ID'

Connect-MgGraph -Environment USGovDoD -TenantId $tenantId -ContextScope Process `
    -Scopes 'Application.Read.All', 'AppRoleAssignment.ReadWrite.All' -NoWelcome
$context = Get-MgContext
if ($context.Environment -ne 'USGovDoD' -or $context.TenantId -ne $tenantId) {
    throw 'Wrong Microsoft Graph cloud or tenant.'
}
$identity = Get-MgServicePrincipal -ServicePrincipalId $identityId -Property
    Id,ServicePrincipalType
if ($identity.ServicePrincipalType -ne 'ManagedIdentity') {
    throw 'The supplied Logic App object ID is not a managed identity.'
}
$api = Get-MgServicePrincipal -ServicePrincipalId $mdeApiObjectId `
    -Property Id,AppRoles,ServicePrincipalNames
$audiences = @($api.ServicePrincipalNames | ForEach-Object { $_.TrimEnd('/') })
if ($audiences -notcontains 'https://api-gov.securitycenter.microsoft.us') {
    throw 'Wrong MDE API audience. Verify the government API registration before proceeding.'
}
foreach ($permission in @('Machine.Isolate', 'Machine.ReadWrite.All')) {
    $role = @($api.AppRoles | Where-Object {
        $_.Value -eq $permission -and $_.IsEnabled -and $_.AllowedMemberTypes -contains
            'Application'
    })
    if ($role.Count -ne 1) { throw "Expected one enabled application role: $permission" }
    $existing = @(Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $identityId -All |
        Where-Object { $_.ResourceId -eq $api.Id -and $_.AppRoleId -eq $role[0].Id })
    if ($existing.Count -eq 0) {
        New-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $identityId -BodyParameter
            @{
            principalId = $identityId
            resourceId = $api.Id
            appRoleId = $role[0].Id
        } | Out-Null
    }
    $verified = @(Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $identityId -All |
        Where-Object { $_.ResourceId -eq $api.Id -and $_.AppRoleId -eq $role[0].Id })
    if ($verified.Count -ne 1) { throw "Permission not verified: $permission. Allow propagation
        and rerun." }
    Write-Output "Verified: $permission"
}
```

`Machine.ReadWrite.All` is broad: MDE documents it for the machine and action-status APIs used by the workflow. Customer approval is required. These MDE permissions are separate from Sentinel workspace permissions.

Phase 4. Connect the Automation Rule & Test

1 **Grant Sentinel Playbook Permissions** *(skip if you already ran Option 3A above)*:

- In **Microsoft Sentinel > Configuration > Settings > Settings > Playbook permissions > Configure permissions**, check the resource group containing your Logic App and click **Apply**.

1 **Create the Automation Rule**:

- In **Microsoft Sentinel > Configuration > Automation > Create > Automation rule**:

Setting	Value
Name	`USB VID_03EB - full isolate`
Trigger	When incident is created

Setting	Value
Condition	Analytics rule name Contains your rule created in Phase 1
Action	Run playbook -> select your Logic App (`isolate-usb-03eb-dod`)
Enabled	On, once ready for pilot/production

1 **Run an End-to-End Pilot or Simulated Test**:

- Either connect an approved test USB device matching `USB\VID_03EB` on a non-critical test machine, **or** run Simulate-UsbTestEvent.ps1 to inject a synthetic `PnpDeviceConnected` test row, verify that the Sentinel Incident is created, the Logic App run succeeds, MDE Action Center reports `Isolate (Full) - Succeeded`, and the incident comment is written.
- Re-run the playbook on the same incident to verify duplicate-request prevention.
- Release the test endpoint from isolation in MDE (or pass `-UnisolateAfterTest` to `Simulate-UsbTestEvent.ps1`).

To stop automated containment at any time: disable the Automation Rule and Analytics Rule, then inspect queued workflow runs and pending MDE actions. Disabling rules does not automatically release already-isolated endpoints.

Monday Customer Call Runbook (15-Minute Agenda)

Use this quick sequence during Monday's live implementation session:

1 **Pre-Flight Check (2 mins)**:

- Confirm the person sharing their screen has **active PIM elevation** for `Privileged Role Administrator` **or** `Global Administrator` (for Entra AppRoles) + `Owner` **or** `RBAC Administrator` on the Sentinel Resource Group.
- Confirm **Microsoft Sentinel > Data connectors > Microsoft Defender XDR** has raw `DeviceEvents` checked.
- Confirm whether the customer wants **Built-in Managed Identity Graph Email** (`notificationSenderEmail` + `notificationRecipientEmail`) or the **Office 365 Outlook connector** in Designer.

1 **Deploy Resources (5 mins)**:

- **If ARM in Portal is approved**: Open **Deploy a custom template**, paste azuredeploy.json, fill in `workspaceName` (and optional email parameters), and click **Create**.
- **If Manual Portal is preferred**: Follow **Phase 1** (paste KQL) and **Phase 2** (create Logic App + Managed Identity + Sentinel trigger connection, then paste portal-workflow-definition.json into `definition` in Code view).
- **If CLI/Cloud Shell Script is approved**: Run Deploy-SentinelUsbIsolation.ps1 with `-CloudEnvironment DoD`.

1 **Grant Permissions in Cloud Shell (3 mins)**:

- Run the **Option 3A** Cloud Shell block above (with `$graphUrl = "https://dod-graph.microsoft.us"` for DoD IL5).
- Authorize the Sentinel API Connection in the Logic App if deployed via ARM.

1 **Live Pilot Test & Sign-off (5 mins)**:

- Run Simulate-UsbTestEvent.ps1 against an approved non-critical test VM (with `-CloudEnvironment DoD -UnisolateAfterTest`) **or** plug in an approved `USB\VID_03EB` test device and run the playbook on the resulting Sentinel incident.
- Verify the MDE Action Center status (`Isolate - Full - Succeeded`), the Sentinel incident comment, and the email notification.

For limits, duplicate handling, logging considerations and Microsoft references,

see the operating notes.